

# **POO 3 (JAVA ET JAVA AVANCÉ)**

**Prof. Nisrine DAD**

**4° Ingénierie Informatique et Réseaux - Semestre I**

Ecole Marocaine des Sciences d'Ingénieur

Année Universitaire : 2024/2025

## V. COLLECTIONS VS STREAMS

## V. COLLECTIONS VS STREAMS

### Les collections

- **Une structure de données** est l'organisation efficace d'un ensemble de données, sous la forme de **tableaux**, de **listes**, de **pires** etc.
- L'**efficacité** d'une structure de données réside dans la **quantité mémoire** utilisée pour **stocker** les données, et le **temps** nécessaire pour réaliser des **opérations sur ces données**.
- **En java**, une collection gère un ensemble d'objets d'un type donné ; ou bien c'est un objet qui sert à stocker un ensemble d'objets d'un type donnée.
- Dans les **premières versions** de Java, les collections étaient représentées par les « **tableau**", "**Vector**", "**Stack**", etc.
- Ces classes sont connues sous le nom **Legacy classes** et elles sont **synchronisées**.

# V. COLLECTIONS VS STREAMS

## Les collections

- Ces classes sont définies dans *java.util*.
- **Exemple: Vector:** Tableau dynamique.

```
* @since 1.0
*/
public class Vector<E>
    extends AbstractList<E>
```

```
* @since 1.0
*/
public class Hashtable<K,V>
    extends Dictionary<K,V>
```

```
* @since 1.0
*/
public class Stack<E> extends Vector<E>
```

```
* @since 1.0
*/
public class Properties extends Hashtable<Object,Object>
```

```
* @since 1.0
*/
public abstract
class Dictionary<K,V>
```

```
public static void main(String[] args) {
    Vector<Integer> vec=new Vector<Integer>();

    vec.add(13);
    vec.add(10);
    vec.add(13);

    for(Integer element :vec)
        System.out.println(element);

    vec.remove(0);

    for(int i=0;i<vec.size();i++)
        System.out.println(vec.get(i));

    System.out.println(vec.capacity());
}
```

```
13
10
13
10
13
10
```

## V. COLLECTIONS VS STREAMS

### Les collections

- Ces anciennes versions utilisent différents noms de méthodes pour accéder, ajouter et manipuler les données.
- Ces classes n'implémentent pas d'interface standard.
- Le framework Collection a été introduit avec la version java 1.2, et les Legacy classes ont été modifiées pour se conformer à ce framework.

```
int arr[] = { 1, 2, 3, 4 };
Vector<Integer> v = new Vector();
Hashtable<Integer, String> h = new Hashtable();

// Ajouter des éléments dans le vector
v.addElement(1);
v.addElement(2);

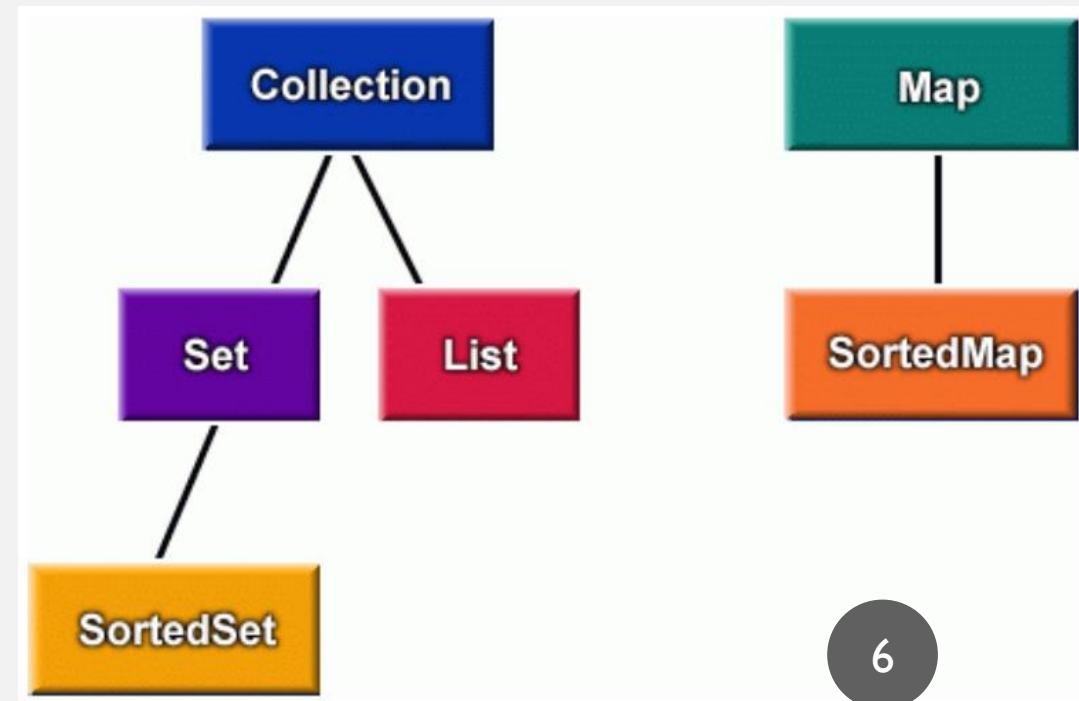
// Ajouter des éléments dans le Hashtable
h.put(1, "EMSI");
h.put(2, "POO");

// Accéder aux éléments du tableau,
// du vector et du hashtable
System.out.println(arr[0]);
System.out.println(v.elementAt(0));
System.out.println(h.get(1));
```

## V. COLLECTIONS VS STREAMS

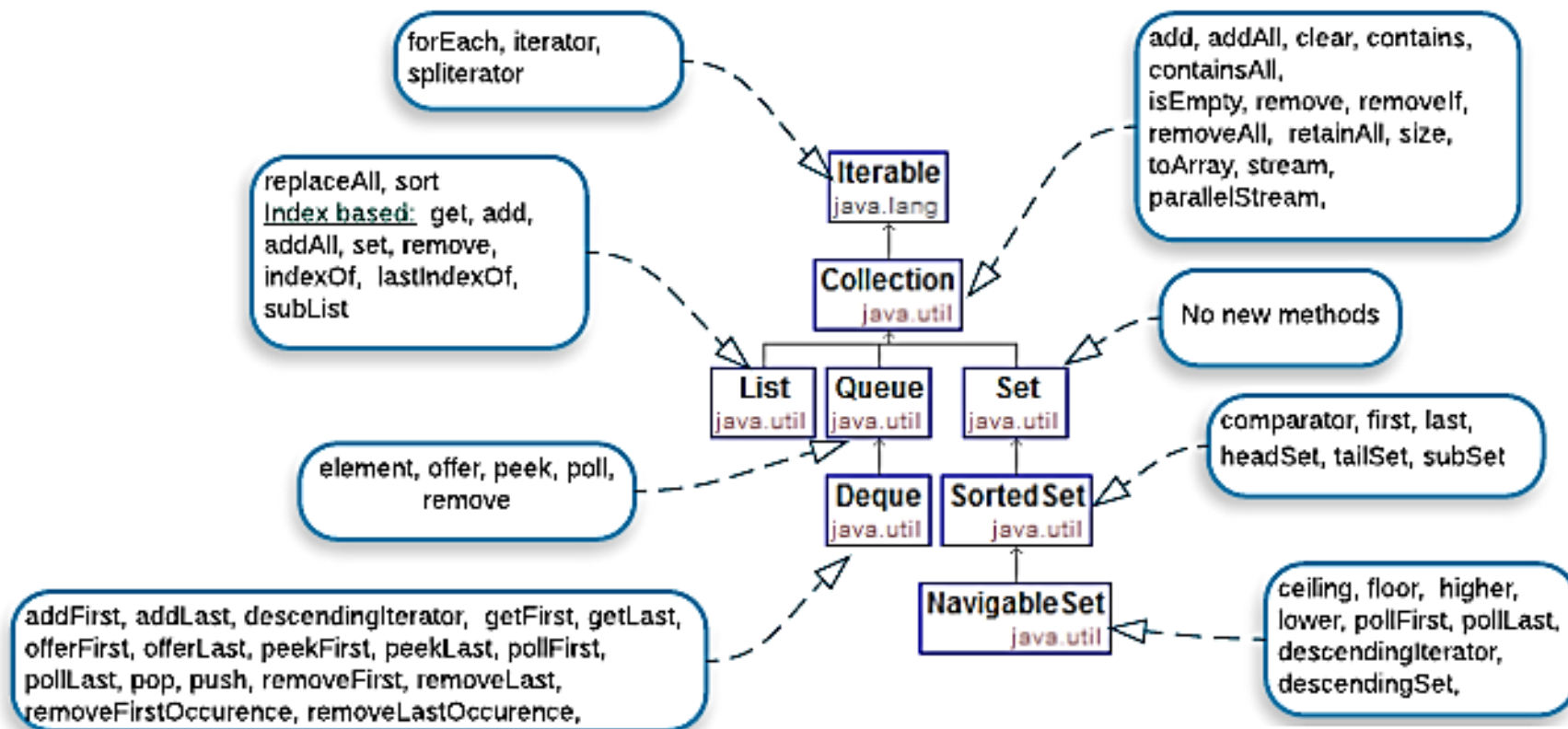
### Les collections

- Avec **Java 1.2** est apparu le **framework de collections** qui tout en gardant les principes de bases, il a apporté des modifications dans la manière avec laquelle ces collections ont été réalisées et hiérarchisées.
- Ce framework est réparti en:
  - **Un ensemble d'interfaces:** Organisées en 2:
    1. Collection,
    2. Map
  - **Un ensemble de classes d'implémentation**



# V. COLLECTIONS VS STREAMS

## collections Methods:



## V. COLLECTIONS VS STREAMS

### Les interfaces de Collections

- **Collection:** un groupe d'objets où la **duplication peut-être autorisée**.
- **Set:** est ensemble ne contenant que des valeurs (**Peut avoir au maximum une valeur null**) et ces **valeurs ne sont pas dupliquées**. Par exemple l'ensemble **A = {1,2,4,8}**. Set hérite donc de Collection, mais n'autorise pas la duplication. **SortedSet** est un Set trié.
- **List:** hérite aussi de Collection, mais **autorise la duplication et donc peut contenir de multiples valeurs null**. Dans cette interface, un système d'indexation a été introduit pour permettre **l'accès (rapide) aux éléments** de la liste avec la méthode *get(int index)*.



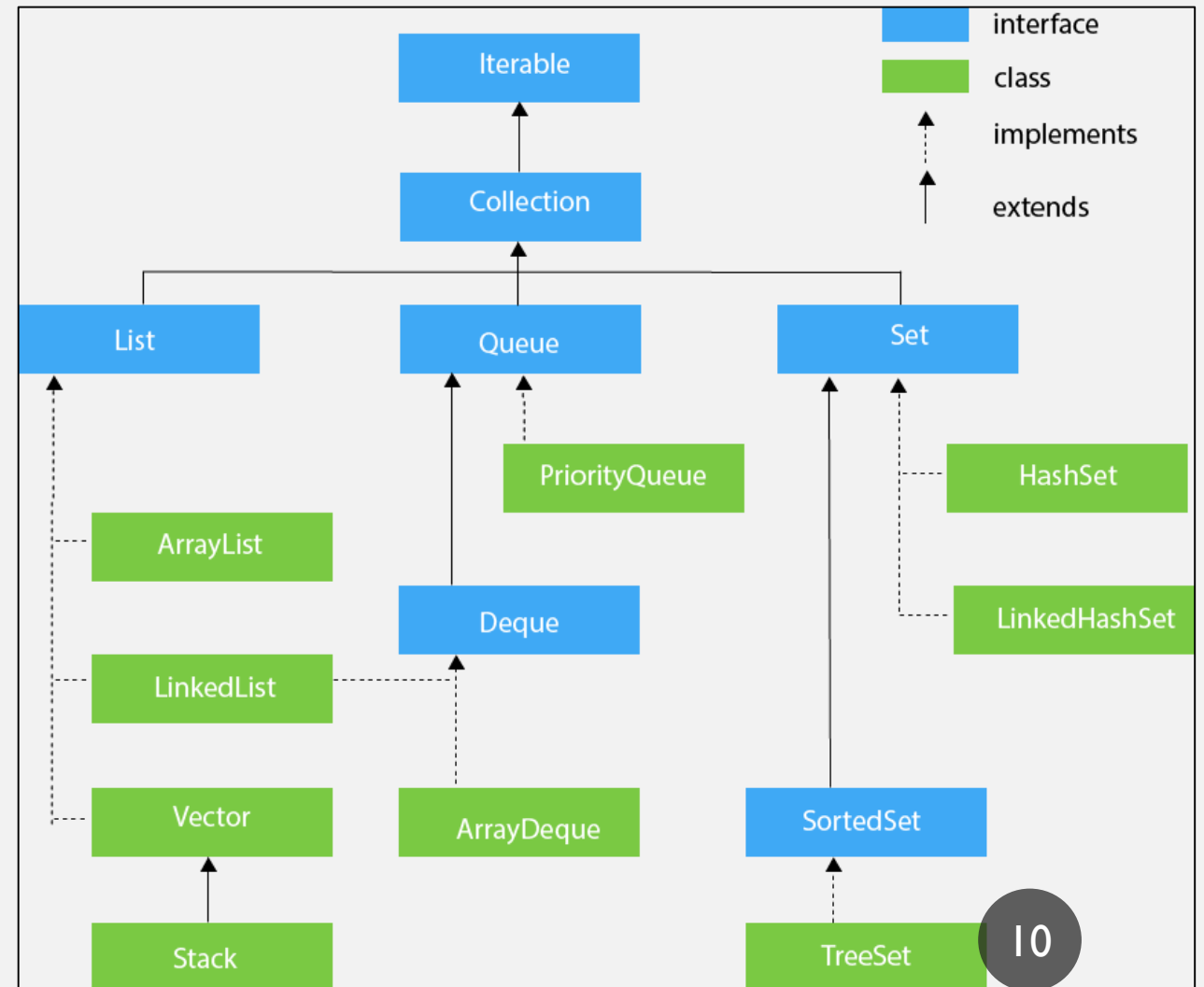
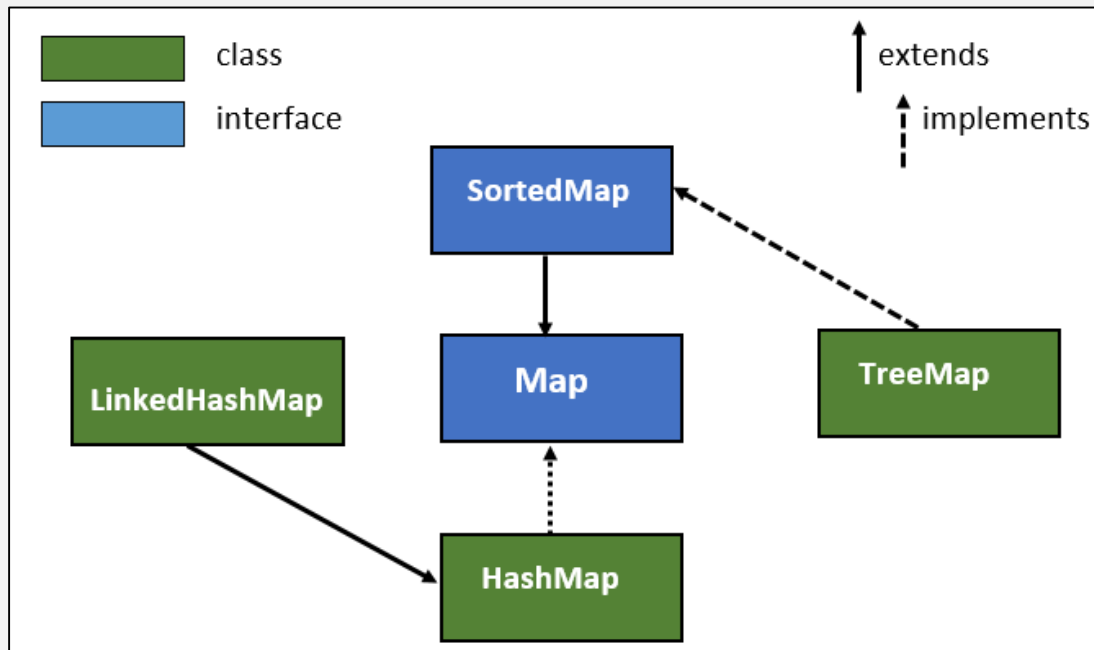
## V. COLLECTIONS VS STREAMS

### Les interfaces de Collections

- **Map:** est un groupe de **paires** contenant une **clé** et une **valeur** associée à cette clé. Cette interface n'hérite ni de Set ni de Collection. La raison est que Collection traite des données simples alors que Map des données **composées (clé,valeur)**. **SortedMap** est un Map trié.

# V. COLLECTIONS VS STREAMS

## Les classes d'implémentation de Collections



## V. COLLECTIONS VS STREAMS

### Les classes d'implémentation de Collections

|            | Classes d'implémentations |                  |                            |               |               |
|------------|---------------------------|------------------|----------------------------|---------------|---------------|
|            |                           | Table de Hachage | Tableau de taille variable | Arbre balancé | Liste chaînée |
| Interfaces | Set                       | HashSet          |                            | TreeSet       |               |
|            | List                      |                  | ArrayList                  |               | LinkedList    |
|            | Map                       | HashMap          |                            | TreeMap       |               |

# Les opérations et itérateurs

- Les **opérations** sont utilisées pour traiter les éléments d'un ensemble de données, par exemple: ajout, suppression, tris, recherche, etc.
- Les **itérateurs** fournissent aux opérations un moyen pour parcourir une collection du début à la fin. Ce moyen permet de récupérer donc, à la demande, des éléments données de la collection.

```
public interface Iterator<E> {  
    boolean hasNext();  
    Object next();  
    void remove();  
}
```

```
public interface Collection<E>  
    extends Iterable<E> {  
    int size();  
    boolean isEmpty();  
    boolean contains(Object o);  
    Iterator<E> iterator();  
    Object[] toArray();  
    <T> T[] toArray(T[] a);  
    boolean add(E e);  
    boolean remove(Object o);  
  
    // Bulk Operations  
    boolean containsAll(Collection<?> c);  
    boolean addAll(Collection<? extends E> c);  
    boolean removeAll(Collection<?> c);  
    boolean retainAll(Collection<?> c);  
    void clear();  
    boolean equals(Object o);  
    int hashCode();  
}
```

## V. COLLECTIONS VS STREAMS

### Les classes d'implémentation de Collections

|            | Classes d'implémentations |                  |                            |               |               |
|------------|---------------------------|------------------|----------------------------|---------------|---------------|
|            |                           | Table de Hachage | Tableau de taille variable | Arbre balancé | Liste chaînée |
| Interfaces | Set                       | HashSet          |                            | TreeSet       |               |
|            | List                      |                  | ArrayList                  |               | LinkedList    |
|            | Map                       | HashMap          |                            | TreeMap       |               |

## V. COLLECTIONS VS STREAMS

### Les opérations et itérateurs de Collections

- Les méthodes d'un itérateur:
  - **hasNext** permet de vérifier s'il y a un élément qui suit.
  - **next** permet de pointer l'élément suivant.
  - **remove** permet de retirer l'élément courant.

```
TreeSet<String> set1 = new TreeSet<String>();  
set1.add("B");  
set1.add("C");  
set1.add("A");
```

```
Iterator<String> i=set1.iterator();  
while (i.hasNext()) {  
    String s= i.next();  
    System.out.println(s);  
}
```

```
// ou bien parcourir avec forEach  
set1.forEach(e->System.out.println(e));
```

A  
B  
C  
A  
B  
C

## V. COLLECTIONS VS STREAMS

### HashSet vs TreeSet

```
HashSet<String> hs = new HashSet<String>();  
// Une table de Hachage  
hs.add("New York City");  
hs.add("Houston");  
hs.add("Tucson");  
hs.add("Los Angeles");  
hs.add("Chicago");  
hs.add("Boston");  
hs.add("Denver");  
hs.add("Chicago");  
hs.add("New York City");  
System.out.println(hs);
```

[New York City, Chicago, Tucson, Los Angeles,  
Denver, Houston, Boston]

```
TreeSet<String> hs = new TreeSet<String>();  
// Un arbre trié  
hs.add("New York City");  
hs.add("Houston");  
hs.add("Tucson");  
hs.add("Los Angeles");  
hs.add("Chicago");  
hs.add("Boston");  
hs.add("Denver");  
hs.add("Chicago");  
hs.add("New York City");  
System.out.println(hs);
```

[Boston, Chicago, Denver, Houston, Los Angeles,  
New York City, Tucson]

## V. COLLECTIONS VS STREAMS

### HashSet

Etudiant [code=2, nom=Karimi]  
Etudiant [code=1, nom=Ahmadi]

```
class Etudiant{
    private int code;
    private String nom;
    public Etudiant(int code, String nom) {
        this.code = code;
        this.nom = nom;
    }
    public String toString() {
        return "Etudiant [code=" + code + ", nom=" + nom + "];"
    }
}

public class Test1 {
    public static void main(String[] args) {
        HashSet<Etudiant> set=new HashSet<Etudiant>();
        set.add(new Etudiant(1,"Ahmadi"));
        set.add(new Etudiant(2,"Karimi"));
        for(Etudiant e:set)
            System.out.println(e);
    }
}
```



## V. COLLECTIONS VS STREAMS

### TreeSet

```
public class Test1 {  
    public static void main(String[] args) {  
        TreeSet<Etudiant> set=new  
        TreeSet<Etudiant>();  
        set.add(new Etudiant(2,"Karimi"));  
        set.add(new Etudiant(1,"Ahmadi"));  
        for(Etudiant e:set)  
            System.out.println(e);  
    }  
}
```

```
Etudiant [code=1, nom=Ahmadi]  
Etudiant [code=2, nom=Karimi]
```

La classe Etudiant doit implementer  
l'interface Comparable

```
class Etudiant implements  
    Comparable<Etudiant>{  
    private int code;  
    private String nom;  
    public Etudiant(int code, String nom) {  
        this.code = code;  
        this.nom = nom;  
    }  
    public String toString() {  
        return "Etudiant [code=" + code + ",  
        nom=" + nom + "];"  
    }  
    public int compareTo(Etudiant o) {  
        if(this.code<o.code)  
            return -1;  
        if(this.code>o.code)  
            return 1;  
        return 0;  
    }  
}
```

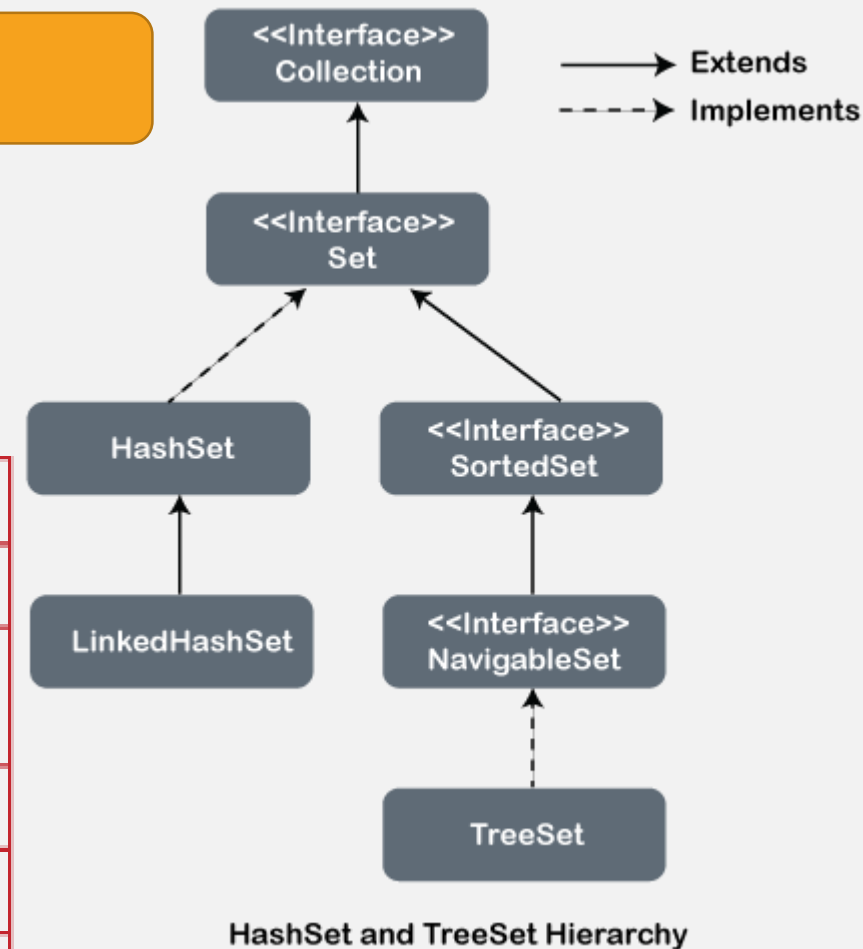
# V. COLLECTIONS VS STREAMS

## HashSet vs TreeSet

- **TreeSet**: les éléments sont rangés de manière ascendante.  
**HashSet**: les éléments sont rangés suivant une méthode de hachage.
- **Les 2** ne permettent pas la duplication.

| Parameters                               | HashSet    | TreeSet        |
|------------------------------------------|------------|----------------|
| Data Structure                           | Hash Table | Red-black Tree |
| Time Complexity<br>(add/remove/contains) | O(1)       | O(log n)       |
| Iteration Order                          | Arbitrary  | Sorted         |
| Null Values                              | Allowed    | Not Allowed    |
| Processing                               | Fast       | Slow           |

HashSet Vs TreeSet



## V. COLLECTIONS VS STREAMS

### ArrayList vs LinkedList

```
ArrayList<String> list = new  
ArrayList<String>();
```

```
list.add("Bernadine");  
list.add("Elizabeth");  
list.add("Gene");  
list.add("Elizabeth");  
list.add("Clara");
```

```
System.out.println(list);
```

```
System.out.println("2: " + list.get(2));  
System.out.println("0: " + list.get(0));
```

```
[Bernadine, Elizabeth, Gene, Elizabeth, Clara]  
2: Gene  
0: Bernadine
```

```
[Bernadine, Elizabeth, Gene, Elizabeth, Clara]  
[Clara, Bernadine, Elizabeth, Gene, Elizabeth, Clara]  
[Clara, Bernadine, Elizabeth, Gene]
```

```
LinkedList<String> listeChaine = new  
LinkedList<String>();
```

```
listeChaine.add("Bernadine");  
listeChaine.add("Elizabeth");  
listeChaine.add("Gene");  
listeChaine.add("Elizabeth");  
listeChaine.add("Clara");  
System.out.println(listeChaine);
```

```
listeChaine.addFirst("Clara");  
System.out.println(listeChaine);
```

```
listeChaine.removeLast();  
listeChaine.removeLast();  
System.out.println(listeChaine);
```

## V. COLLECTIONS VS STREAMS

### ArrayList

- Basée sur un tableau dynamique, ce qui la rend efficace pour les opérations d'accès aléatoire.
- Les opérations d'ajout ou de suppression d'éléments à la fin sont rapides, mais l'insertion ou la suppression d'éléments au milieu peut être coûteuse car cela nécessite de décaler les éléments.
- Non synchronisée par défaut. Utilisez **Collections.synchronizedList** si une synchronisation est nécessaire.
- Plus appropriée lorsque vous effectuez fréquemment des accès aléatoires aux éléments de la liste.

## V. COLLECTIONS VS STREAMS

### LinkedList

- Basée sur une liste doublement chaînée, ce qui la rend efficace pour les opérations d'insertion ou de suppression au milieu de la liste.
- L'accès aléatoire est plus coûteux car il nécessite un parcours séquentiel de la liste.
- Non synchronisée par défaut. Utilisez **Collections.synchronizedList** si une synchronisation est nécessaire.
- Plus appropriée lorsque vous effectuez fréquemment des insertions ou des suppressions au milieu de la liste.

## V. COLLECTIONS VS STREAMS

### Vector

- Une implémentation ancienne de List qui est entièrement synchronisée, ce qui signifie qu'elle est thread-safe.
- Moins utilisée de nos jours en raison de sa synchronisation, qui peut entraîner des performances moins bonnes dans un contexte multithread.
- Les opérations d'accès, d'ajout ou de suppression sont similaires à celles de ArrayList.
- Utilisez plutôt ArrayList ou Collections.synchronizedList pour une synchronisation sélective.

## V. COLLECTIONS VS STREAMS

### Exercices

- Écrivez un programme Java pour:
- créer un ArrayList nommé « languages », ajoutez des chaînes (Ex: PHP, Java, C++, Python) et affichez la collection.
- parcourir tous les éléments de la liste, en utilisant la boucle for normale, for améliorée et l'itérateur.
- insérer l'élément « Pascal » en première position dans la liste.
- récupérer le troisième élément à partir de la liste.
- mettre à jour le troisième élément de la liste par « COBOL »
- supprimer le troisième élément de la liste.
- rechercher un élément dans la liste.
- Trier la liste.

```
ArrayList<String> languages=new  
ArrayList<String>();  
  
languages.add("PHP");  
languages.add("Java");  
languages.add("C++");  
languages.add("Python");  
  
System.out.println(Languages);  
  
for(String element:languages)  
    System.out.print(element+" ");  
for(int i=0;i<languages.size();i++)  
    System.out.print(Languages.get(i)+" ");  
  
Iterator<String> i=languages.iterator();  
while(i.hasNext()) {  
    System.out.print(i.next()+" ");  
}
```

```
languages.add(0, "Pascal");  
  
System.out.println(Languages.get(2));  
  
languages.set(2, "COBOL");  
  
languages.remove(2);  
  
System.out.println(Languages.contains("COBOL"));  
  
Collections.sort(Languages);
```



## V. COLLECTIONS VS STREAMS

### Map

- C'est un ensemble de paires, contenant une clé et une valeur.
- Deux clés ne peuvent être égales au sens de **equals**.

## V. COLLECTIONS VS STREAMS

### HashMap

- Basée sur le principe du hachage.
- Performance moyenne constante pour les opérations de base (get et put) en moyenne, sous réserve d'une bonne fonction de hachage et d'une distribution correcte des éléments.
- Ne garantit pas un ordre spécifique des éléments.
- Autorise les clés et les valeurs null.
- Non synchronisée par défaut. Utilisez **Collections.synchronizedMap** pour obtenir une version synchronisée si nécessaire.

## V. COLLECTIONS VS STREAMS

### TreeMap

- Basée sur un arbre Red-Black (arbre rouge-noir).
- Maintient les éléments dans un ordre trié, basé sur leur ordre naturel ou un comparateur spécifié.
- Complexité temporelle de  $\log(n)$  pour les opérations de base, ce qui la rend généralement plus lente que HashMap pour des ensembles de données importants.
- N'autorise pas les clés null, mais autorise les valeurs null.
- Non synchronisée par défaut. Utilisez **Collections.synchronizedSortedMap** pour obtenir une version synchronisée si nécessaire.

## V. COLLECTIONS VS STREAMS

### HashTable

- Une ancienne implémentation de la structure de données de hachage, introduite bien avant HashMap.
- Similaire à HashMap mais est complètement synchronisée, ce qui signifie qu'elle est thread-safe.
- N'autorise pas les clés ou les valeurs null.
- Déconseillée dans les nouveaux développements.

## V. COLLECTIONS VS STREAMS

### Map

```
TreeMap<Integer,String> hm=new  
TreeMap<Integer,String>();
```

```
hm.put(0, "Etudiant0");  
hm.put(3, "Etudiant2");  
hm.put(1, "Etudiant1");  
hm.put(2, "Etudiant2");  
hm.put(3, "Etudiant3");
```

```
hm.remove(2);  
System.out.println(hm.get(1));
```

```
System.out.println(hm);
```

```
Etudiant1  
{0=Etudiant0, 1=Etudiant1, 3=Etudiant3}
```

```
HashMap<Integer,String> hm=new  
HashMap<Integer,String>();
```

```
hm.put(0, "Etudiant0");  
hm.put(3, "Etudiant2");  
hm.put(1, "Etudiant1");  
hm.put(2, "Etudiant2");  
hm.put(3, "Etudiant3");
```

```
hm.remove(2);  
System.out.println(hm.get(1));
```

```
System.out.println(hm);
```

```
Etudiant1  
{0=Etudiant0, 1=Etudiant1, 3=Etudiant3}
```

## V. COLLECTIONS VS STREAMS

### Les streams

- L'API streams a été introduite avec Java 8 pour permettre la programmation fonctionnelle.
- Un stream (flux) est une représentation d'une séquence sur laquelle il est possible d'appliquer des opérations. Cette API a deux principaux intérêts :
- Un stream permet d'effectuer les opérations sur une séquence **sans utiliser de structure de boucle**. Cela permet de réaliser des traitements complexes tout en maintenant une **bonne lisibilité du code**.

## V. COLLECTIONS VS STREAMS

### Création d'un stream

- On peut créer des streams de plusieurs manières. Cependant, on va se limiter dans ce chapitre à 2 types de créations:
  - **Création à partir d'un tableau**
  - **Création à partir d'une collection.**

```
int[] tableau = { 1, 2, 3, 4 };  
IntStream tableauStream = Arrays.stream(tableau);  
  
List<Integer> a = Arrays.asList(1, 2, 2,3,4);  
Stream<Integer> s=a.stream();
```

## V. COLLECTIONS VS STREAMS

### Les streams: La réduction

- La réduction consiste à obtenir un **résultat unique** à partir d'un stream.
- On peut par exemple compter le nombre d'éléments.
- Si le stream est composé de nombres, on peut réaliser une réduction mathématique en calculant la **somme**, la **moyenne** ou en demandant la valeur **minimale** ou **maximale**...

```
int t[] = {1,2,3};  
System.out.println(Arrays.stream(t).count());  
System.out.println(Arrays.stream(t).min());  
System.out.println(Arrays.stream(t).max());  
System.out.println(Arrays.stream(t).sum());  
System.out.println(Arrays.stream(t).average());  
System.out.println(L.stream().reduce((x,y)->x*y));
```

```
3  
OptionalInt[1]  
OptionalInt[3]  
6  
OptionalDouble[2.0]  
6
```



## V. COLLECTIONS VS STREAMS

### Les streams: La collecte

- La collecte permet de créer une nouvelle collection à partir d'un stream.
- Pour cela, il faut fournir une implémentation de l'interface `Collector`.
- Cette interface est assez complexe, heureusement la **classe utilitaire `Collectors`** fournit des méthodes pour générer une instance de `Collector`. Pour réaliser la collecte, il faut appeler la méthode **`collect()`**.

```
List<Integer> a = Arrays.asList(1, 2, 3);  
List<Integer> liste = a.stream().collect(Collectors.toList());  
System.out.println(liste);  
List<String> s = Arrays.asList("Hello", "world");  
System.out.println(s.stream().collect(Collectors.joining(",")));
```

```
[1, 2, 3]  
Hello,world
```

## V. COLLECTIONS VS STREAMS

### Les streams: Le filtrage

- Une opération courante sur un stream consiste à appliquer un filtre pour éliminer une partie de ses éléments. Pour, cela on peut utiliser la méthode **filter()** qui prend en paramètre un **Predicate**.

```
List<Integer> a = Arrays.asList(1, 2, 3);  
a.stream().filter(e->e!=2).forEach(System.out::println);
```

```
1  
3
```

## V. COLLECTIONS VS STREAMS

### Les streams: Le mapping

- Le mapping est une opération qui permet de transformer la nature du stream afin de passer d'un type à un autre.
- Pour réaliser un mapping vers un type primitif, il faut utiliser les méthodes `Stream.mapToInt`, `Stream.mapToLong` ou `Stream.mapToDouble`.
- On peut également utiliser ces méthodes pour convertir un stream contenant un type primitif vers un stream contenant un autre type primitif.

```
List<Integer> a = Arrays.asList(1, 2, 3);

System.out.println(a.stream().min((x,y)->{return x-y;}));
System.out.println(a.stream().max((x,y)->{return x-y;}));
System.out.println(a.stream().mapToInt(e->e).min());
System.out.println(a.stream().mapToInt(e->e).max());
System.out.println(a.stream().mapToInt(e->e).sum());
System.out.println(a.stream().mapToInt(e->e).average());
IntSummaryStatistics stats = a.stream().mapToInt(e -> e).summaryStatistics();
System.out.println(stats);
System.out.println(a.stream().map(e -> e.toString()).collect(Collectors.joining("/")));
```

```
Optional[1]
Optional[3]
OptionalInt[1]
OptionalInt[3]
6
OptionalDouble[2.0]
IntSummaryStatistics{count=3, sum=6, min=1, average=2,000000, max=3}
1/2/3
```

## V. COLLECTIONS VS STREAMS

### Les streams: Exercice

- Supposons une classe `Personne` caractérisée par le nom, l'âge et la ville.
- Écrivez un programme qui utilise des streams pour effectuer les opérations suivantes sur une liste de personnes:
- Filtrer les personnes dont l'âge est supérieur à 25 et mapper ces personnes filtrées pour obtenir une liste de leurs noms.
- Trier les personnes par âge (du plus jeune au plus vieux), puis limiter la liste aux 3 premières personnes après le tri et mapper ces personnes pour obtenir une liste de leurs villes.